

Pecan User Documentation

Contents

1	Introduction	2
2	Built-in functionality	3
2.1	Pecan	3
2.1.1	Expressions	3
2.1.2	Types	3
2.1.3	Predicates	4
2.1.4	Annotations	6
2.1.5	Directives	8
2.2	Praline	9
2.2.1	Built-in methods	10
2.2.2	Supplementary methods	12

Chapter 1

Introduction

Pecan is an automatic theorem prover based on Büchi automata, by utilizing the open source C++ library *Spot* to decide the truth value of predicates. This document serves as documentation for the features it provides, to make it more usable for anyone without the need to scour the source code.

Although an effort is made to be as clear as possible in the definitions of this documentation, this is not always going to work for everyone. Notably, code examples are out-of-scope for this, hence it is recommended to take a look at the standard library if any questions arise.

If there is anything notable missing from the documentation, or something is phrased in an unclear or even wrong way, please open an issue or pull request on the Github repo.

Chapter 2

Built-in functionality

Pecan provides various features built-in, no matter if you have loaded any libraries or not. This built-in functionality can be separated into two distinct parts, the features that are part of Pecan, and those which are technically part of the Praline scripting language.

2.1 Pecan

This section is about all things defined by or definable within Pecan, even if the standard library is not imported.

2.1.1 Expressions

An *expression* is either a numerical constant, a variable, or an arithmetic combination of these. Since a completely rigorous formulation of these in first order logic gets complicated rather fast, Pecan provides certain constructs to make them easy to write:

Arithmetic expressions

Expressions can use addition (+), subtraction (−), multiplication by an integer constant (*) and division by an integer constant (/).

Constants

Expressions can use any integer constant, which Pecan infers from 0 and 1 via repeated addition. If no predicate for 1 is given it is implicitly defined as the smallest value larger than 0, assuming such a value exists.

2.1.2 Types

Variables in Pecan always have types associated with themselves, which are used for resolving arithmetic expressions.

Restrictions

You can globally restrict the type of variables by using:

`Restrict <variables> are <predicate name> .`

wherein the variables are in a comma-separated list. If the predicate only has one argument only the predicate name should be written, otherwise all but the last argument need to be specified in parenthesis.

Global restrictions are additive, so you can apply multiple restrictions on a variable. However only the last is used for arithmetic resolving, while all previous restrictions are merely kept for quantifiers.

If a global restriction is applied to a variable within an imported file, that restriction will not be carried over into your file.

Structures

To use types in resolving arithmetic expressions, it is necessary to define a structure.

`Structure <predicate name> defining <python dictionary>`

The same rules as in type restrictions apply here for the predicate name. The python dictionary is simply a mapping from names to predicates, formatted the following way:

`"<name>": <predicate name> ( <variables> )`

with each entry separated by a comma.

Note that variables used in the type can be used in the variable lists for predicates in the structure. Furthermore, unlike the type restriction, the predicates must list every variable. The variables that get replaced during arithmetic resolving should simply be written as **any** by convention.

The following names are used by default for arithmetic expressions:

- `adder` for `+`
- `less` for `<`
- `equal` for `=`
- `zero` for `0`
- `one` for `1`

Any of these that is not explicitly defined in a structure will be resolved using the respective predicates used by the natural numbers, assuming the standard library is loaded.

2.1.3 Predicates

A *predicate* is a statement that can either be always true, always false, or sometimes true. Predicates are usually built as a conjunction and / or disjunction of various other predicates, for which Pecan provides a notational standard:

Comparisons

Comparisons combine two expressions into a predicate. They are written using infix notation with any of the following operator symbols:

- $=$
- $\neq$, $\neg=$, $\sim=$, $\neq$
- $>$
- $\geq$, $\geq$
- $<$
- $\leq$, $\leq$

Conjunctions

Conjunctions combine two predicates into one, which is true if and only if both contained predicates are true. They are written using infix notation with any of the following operator symbols:

- and
- $\&$
- $/\wedge$
- $\wedge$

Disjunctions

Disjunctions combine two predicates into one, which is true if and only if at least one of the contained predicates is true. They are written using infix notation with any of the following operator symbols:

- or
- $|$
- $\vee$
- $\vee$

Complements

Complements invert a predicate, being true if and only if the contained predicate is false. They are written using prefix notation with any of the following operator symbols:

- not
- $!$
- $\sim$
- $\neg$

Quantifiers

Quantifiers add restrictions on the existence of variables used within a predicate. They are written using prefix notation in the following form:

```
<quantifier> <variables> .
```

with the variables comma-separated. You can also use the same notation as in global type restrictions to restrict the variables to fulfill that predicate.

```
<quantifier> <variables> are <predicate name> .
```

Note that if a variable has global type restriction, it will be implicitly included when evaluating a quantifier.

There are two quantifiers available in Pecan:

- `exists`, $\exists$
- `forall`, $\forall$

Named predicates

For many purposes, including saving, defining other predicates, or defining types, you need to assign a name to a predicate. This can be done by writing:

```
<predicate name> ( <variables> ) := <predicate>
```

wherein the variables are in a comma-separated list.

LTL formulas

Pecan provides the ability to parse *Linear Temporal Logic* (LTL) formulas directly via Spot, which can be used as predicates in Pecan formulas.

```
"<LTL formula>"
```

Note that LTL formulas require use of Spot syntax. See <https://spot.lre.epita.fr/tl.pdf> for reference.

2.1.4 Annotations

You can wrap any predicate in an annotation to affect how Pecan evaluates the contained predicate. All annotations are declared with the following notation:

```
@<annotation name> [ <predicate> ]
```

`no_simplify`

The `no_simplify` annotation sets the internal simplification level to 0. This reduces the amount of attempted simplifications and postprocessing to a minimum.

`simplify`

The `simplify` annotation sets the internal simplification level to 1. This applies the default level of attempted simplifications and postprocessing.

simplify_high

The **simplify_high** annotation sets the internal simplification level to 2. This increases the amount of attempted simplifications and postprocessing to the maximum.

postprocess_low

The **postprocess_low** annotation applies postprocessing on the contained predicate. This postprocessing is called with the **low** argument, meaning only a small amount of optimizations are attempted.

postprocess_medium

The **postprocess_medium** annotation applies postprocessing on the contained predicate. This postprocessing is called with the **medium** argument, meaning several optimizations are attempted and chained together.

postprocess_high

The **postprocess_high** annotation applies postprocessing on the contained predicate. This postprocessing is called with the **high** argument, meaning many optimization strategies are attempted and the best result is returned.

postprocess

The **postprocess** annotation applies postprocessing on the contained predicate. This postprocessing is not called with any explicit argument, and hence defaults to **high** unless the **--heuristics** flag is used

simplify_states

The **simplify_states** annotation attempts to reduce the number of states of the contained predicate.

simplify_edges

The **simplify_edges** annotation attempts to reduce the number of edges of the contained predicate.

merge_states

The **merge_states** annotation merges redundant states of the contained predicate, if there are any. This is implicitly called when using **simplify_states**.

merge_edges

The **merge_edges** annotation merges redundant edges of the contained predicate, if there are any. This is implicitly called when using **simplify_edges**.

merge_edges_loop

The **merge_edges_loop** annotation repeatedly merges redundant edges of the contained predicate until none are present anymore

2.1.5 Directives

Pecan provides a number of special commands, called *Directives*. These serve a variety of purposes and can be used with the following notation:

```
#<directive name> ( <arguments> )
```

import

The `import ( "<filename>" )` directive loads a given file, making all definitions from that file available. Unless explicitly disabled with the `--no-stdlib` flag, the main file `std.pn` of the standard library is always implicitly imported.

load

The `load ( "<filename>" , "<format>" , <predicate name> ( <arguments> ) )` directive loads an automaton from a given file and format.

save__aut

The `save__aut ( "<filename>" , <predicate name> )` directive saves an automaton in HOA format in the given file. Although not binding, the convention used by Pecan is to have this filename use the `.aut` file extension.

save__aut__image

The `save__aut__image ( "<filename>" , <predicate name> )` directive saves a graphical depiction of an automaton in the given file. This depiction is a vector graphic using the `.svg` file format

context

The `context ( "<context name>" , "<predicate name>" )` directive associates a predicate to a given name. This predicate effectively acts as a "fallback" or "generic" value for that name, being used if no predicate sharing that name exists.

end__context

The `end__context ( "<context name>" )` directive removes a previously established context name.

forget

The `forget ( <variable name> )` directive removes all previously established global restrictions on a given variable name.

assert__prop

The `assert__prop ( <truth value> , <predicate name> )` directive evaluates a given predicate and checks if it holds the given truth value. This truth value can be either `true`, `false`, or `sometimes`. Note that this check is strict, i.e. `#assert__prop (sometimes, <predicate name>)` will fail if that predicate is always true.

shuffle

The `shuffle ( <predicate name> , <predicate name> , <predicate name> )` directive computes the shuffle automaton of the first two named predicates and saves it under the third predicate name.

shuffle_or

The `shuffle_or ( <predicate name> , <predicate name> , <predicate name> )` directive computes the disjoint shuffle automaton of the first two named predicates and saves it under the third predicate name.

2.2 Praline

Praline is a scripting language built into Pecan, allowing more complicated outputs, formatting, and macros. For this, it adds specific variable types and some built-in methods.

Types

There are 5 base types which a value in Praline can assume.

PralineInt

PralineInt is the sole integer type for Praline. It can assume any value that a Python `int` can, i.e. they are signed and can be effectively arbitrarily large.

PralineString

PralineString is the type handling all text strings or characters for Praline. It can be created by wrapping text in a double quotation mark "`<text>`".

PralineBool

PralineBool is the boolean truth value type for Praline. It can either be `true` or `false`, all lowercase.

PralineTuple

PralineTuple is the fixed size container type for Praline. It can be created by wrapping comma-separated values in parenthesis (`<value 1>`, `<value 2>`, ...). Note that tuples require at least two elements!

PralineList

PralineList is the dynamic size container type for Praline. It can be created by wrapping comma-separated values in square brackets [`<value 1>`, `<value 2>`, ...]. These can be created with arbitrarily many elements

PralinePecanTerm

PralinePecanTerm is a Pecan wrapper type for Praline. It can hold any Pecan term, be it a predicate, an expression, or directive, by wrapping it in curly brackets {<pecan term>}.

PralineAutomatonBuilder

PralineAutomatonBuilder is a factory object to create automaton for Praline. It can be created and modified by using built-in methods.

2.2.1 Built-in methods

Pecan provides several built-in methods to manipulate Praline variables. These are always loaded, even if the standard library is not.

truthValue

truthValue <pecan term> returns the truth value of a **PralinePecanTerm**. The outputs can therefore be **true**, **false**, or **sometimes**.

toString

toString <any> returns a **PralineString** containing the input value.

print

print <any> outputs the string representation of the input via the logger, then returns **true**.

emit

emit <pecan term> outputs a string representation of the input Pecan term, then returns **true**. If the term is a predicate definition or directive, it will also get processed by Pecan.

freshVar

freshVar returns a unique string that can act as an unused variable name. Repeated calling of this method will yield a new string every time.

toChars

toChars <string> returns a list consisting of strings of the individual characters making up the input. Non-string values are not accepted, but may be used if first combined with **toString**

cons

cons <head> <tail> returns a list consisting of the head and tail values. If <tail> is a list, each element of this list will get appended separately, instead of being a single entry.

enumFromTo

enumFromTo <low> <high> returns a list consisting of all integer values from **low** to **high**, endpoints included. This can also be called by writing [<low>..**<high>**].

acceptingWord

`acceptingWord <pecan term>` returns a list mapping variable names to accepted words of the given predicate. Each entry of this list is a tuple consisting of the variable name and a binary LSD representation of the prefix and cycle of the accepting word. Note that this is merely the internal representation of the value, and may not be easily human-readable!

compare

`compare <left> <right>` returns whether the left value is larger, smaller, or equal to the right value. If it is larger, it returns 1, if it is smaller it returns -1, and if the values are equal it returns 0.

equal

`equal <left> <right>` returns whether the two values are equal or not. This works with all combinations of types!

mkAut

`mkAut <input names> <input bases>` returns a `PralineAutomatonBuilder` with the given input names and bases. The input names should be a list of strings, while the bases are a list of integers.

addState

`addState <automaton> <state name> <acceptance>` returns the given automaton with the specified state added. If the `<acceptance>` is `true`, this will be set to be an acceptance state.

addTransition

`addTransition <automaton> <source> <destination> <accepted values>` returns the given automaton with the specified transition added. Note that the accepted values are given as a string containing space-separated values.

buildAut

`buildAut <automaton>` returns a `PralinePecanTerm` containing the built automaton of the given `PralineAutomatonBuilder` object.

writeFile

`writeFile <filepath> <content>` writes the contents to a file at the given filepath, then returns `true`.

readFile

`readFile <filepath>` returns the contents of a file at the given filepath.

deleteFile

`deleteFile <filepath>` deletes the file at the given filepath, then returns `true`.

splitStr

`splitStr <separator> <input>` returns a list of substrings of the given input, using the given separator.

plot

`plot <options> <numeration systems> <pecan term>` plots the automaton contained inside the Pecan term using the given options and numeration systems, then returns `true`. Both the options and numeration systems should be lists of tuples. Note: This is primarily intended for displaying fractals, if you want an image of the given automaton use the `save_aut_img` directive instead!

set

`set <setting> <value>` sets the value of the setting to the given value, then returns `true`. Currently the following settings can be set with this method:

- `output_json`
- `show_progress`
- `write_statistics`
- `extract_implications`
- `min_opt`
- `simplification_level`
- `history_file`
- `debug_level`
- `quiet`
- `opt_level`
- `heuristics`
- `postprocessing_preference`
- `postprocessing_force_sbacc`
- `load_stdlib`
- `output_hoa`

2.2.2 Supplementary methods

Aside from the built-in methods which are used for actually interfacing with data, Praline also has some supplementary methods for working with regular methods. Keep in mind that, unlike Pecan directives, the calls for these always end with a period!

Execute

Execute <praline term> . is the Praline directive that causes any method given as argument to be executed. It is the only built-in Praline directive.

Define

Define <operator> := <term> . is the method used to define new methods during runtime.

Alias

Alias <string> ==> <praline directive> <praline term> . defines a new Praline directive with the given alias. Note that, since **Execute** is the only built-in Praline directive, all Praline directives defined at runtime will start with it, and hence can be used without explicitly calling it.